

ФКН ВШЭ
Глубинное обучение 1, 2024/25

Автор конспекта: Даниил Тихонов

27 марта 2025 г. в 21:54

Содержание

1. Диффузионные модели	1
1.1. Denoising Diffusion Probabilistic Models (DDPM)	1
1.2. Обыкновенные дифференциальные уравнения	4
1.3. Стохастические дифференциальные уравнения	5
1.4. Обращение времени	5
1.5. Обобщение DDPM на непрерывное время	6
1.6. Denoising score matching	6

Лекция #15: Диффузионные модели

3 марта, 2025

1.1. Denoising Diffusion Probabilistic Models (DDPM)

Если в самых общих словах описывать диффузионные модели, то идея следующая: возьмем какую-то картинку и будем постепенно на каждом шаге добавлять в нее стандартный нормальный шум, пока она полностью не превратится в шум. Затем обучим модель, которая, наоборот, будет убирать из нее немного шума на каждом шаге, пока не получится картинка.

Предположим, что наши данные являются векторами $x \in \mathbb{R}^n$. Теперь для каждого момента времени $t \in [0, T] \cap \mathbb{Z}$ зададим вероятностное распределение на x таким образом, что

- $x_0 \sim p_{data}$, где p_{data} – распределение реальных данных
- $x_T \sim \mathcal{N}(0, I)$
- $q(x_t | x_{t-1}) := \mathcal{N}(\sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$, где β_t – это какое-то маленькое число, зависящее от времени t (в простейшем случае можно думать, что $\beta_t \equiv \beta$ – константа).

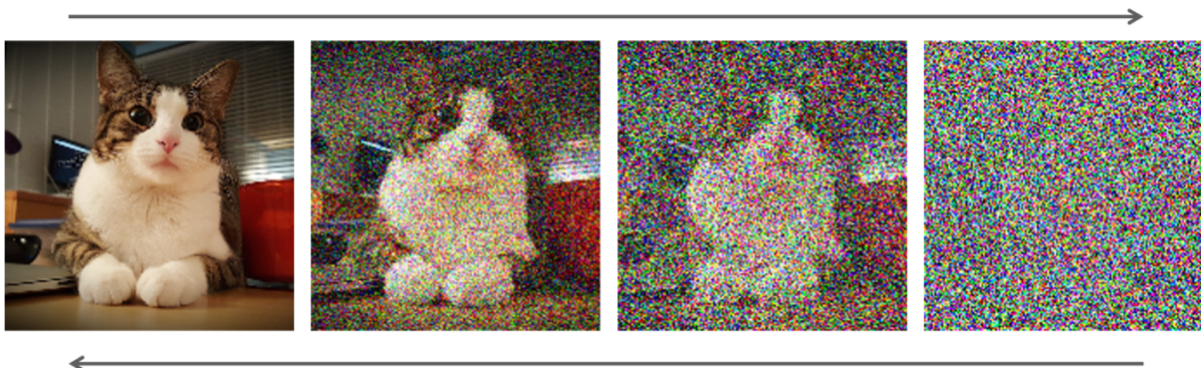
В данной модели $q(x_t | x_{t-1})$ задает некоторый марковский процесс или же правило, по которому данные переводятся в шум в каждый момент времени t .

Задаваемый $q(x_t | x_{t-1})$ процесс называется **прямым процессом**.

Запишем совместное распределение всей траектории:

$$q(x_{1:T} | x_0) := q(x_1, \dots, x_T | x_0) = \prod_{t=1}^T q(x_t | x_{t-1})$$

То есть пока все достаточно просто: картинка зашумляется, информация из нее исчезает, энтропия увеличивается. Нейросеть мы будем учить разворачивать этот прямой процесс, то есть получать на вход шум $z \in \mathcal{N}(0, I)$ и делать обратные шаги (расшумлять картинку).



Зададим эту модель с параметрами θ следующим образом:

$$p_\theta(x_{t-1} | x_t) := \mathcal{N}(x_{t-1} | \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$$

На самом деле, в общем случае матрицу ковариации можно не обучать, и просто полагать

$$p_\theta(x_{t-1} | x_t) := \mathcal{N}(x_{t-1} | \mu_\theta(x_t, t), \beta_t I)$$

В таком сетапе совместное распределение всей траектории имеет вид:

$$p_\theta(x_{0:T}) := p_\theta(x_0, \dots, x_T) = p(x_T) \prod_{t=1}^T p_\theta(x_{t-1} | x_t), \quad p(x_T) \sim \mathcal{N}(0, I)$$

Предположим, что мы уже справились обучить модель p_θ . Тогда ниже представлен алгоритм семплирования из нее новых данных:

- Семплируем $x_T \sim \mathcal{N}(0, I)$
- Для каждого $t = 1, \dots, T$ семплируем $x_{t-1} \sim p_\theta(x_{t-1} | x_t)$
- Получаем сгенерированный объект x_0

Заметим, что на каждом шаге семплирования из пункта (2) картинка постепенно расшумляется, но при этом в нее все равно добавляется немного шума из-за дисперсии $\beta_t I$.

Конечной целью является возможность семплировать из распределения данных, то есть

$$x_0 \sim p_{data}$$

Поэтому учить модель мы будем, максимизируя правдоподобие сгенерированного объекта:

$$\begin{aligned} \log p_\theta(x_0) &= \log \int p_\theta(x_{0:T}) dx_1 \dots dx_T = \log \int q(x_{1:T} | x_0) \frac{p_\theta(x_{0:T})}{q(x_{1:T} | x_0)} dx_1 \dots dx_T = \\ &= \log \mathbb{E}_{q(x_{1:T} | x_0)} \left[\frac{p_\theta(x_{0:T})}{q(x_{1:T} | x_0)} \right] \geq \mathbb{E}_{q(x_{1:T} | x_0)} \left[\log \frac{p_\theta(x_{0:T})}{q(x_{1:T} | x_0)} \right] \longrightarrow \max_{\theta} \end{aligned}$$

В **переходе выше** мы воспользовались неравенством Йенсена для логарифма. Как и во многих других моделях, будем максимизировать нижнюю оценку, таким образом косвенно максимизируя исходное правдоподобие. Распишем нижнюю оценку, используя формулы выше:

$$\begin{aligned} \mathbb{E}_{q(x_{1:T} | x_0)} \left[\log \frac{p_\theta(x_{0:T})}{q(x_{1:T} | x_0)} \right] &= \mathbb{E}_{q(x_{1:T} | x_0)} \left[\frac{p(x_T) \prod_{t=1}^T p_\theta(x_{t-1} | x_t)}{\prod_{t=1}^T q(x_t | x_{t-1})} \right] = \\ &= \mathbb{E}_{q(x_{1:T} | x_0)} \left[\log p(x_T) + \sum_{t=1}^T \frac{p_\theta(x_{t-1} | x_t)}{q(x_t | x_{t-1})} \right] =: \mathcal{L}_{oss} \end{aligned}$$

В явном виде этот интеграл считать трудно, так как матожидание берется по какому-то сложному распределению. Вместо этого будет приближать лосс методом Монте-Карло:

- Берем обучающий объект x_0 из датасета
- Семплируем для него траекторию прямого процесса $x_1 \sim q(x_1 | x_0), \dots, x_T \sim q(x_T | x_{T-1})$
- Считаем $\nabla_{\theta} \left(\sum_{t=1}^T \frac{p_{\theta}(x_{t-1} | x_t)}{q(x_t | x_{t-1})} \right)$ и обновляем параметры θ

В такой постановке возникает проблема, что для одного шага градиентного спуска нужно семплировать всю траекторию (сгенерировать T картинок), что очень долго.

На практике матожидание из лосса на самом деле можно уточнить:

$$\mathbb{E}_{q(x_{1:T} | x_0)} \left[-\text{KL} (q(x_T | x_1) \parallel p(x_T)) - \sum_{t=1}^T \text{KL} (q(x_{t-1} | x_t, x_0) \parallel p_{\theta}(x_{t-1} | x_t)) + \log p_{\theta}(x_0 | x_1) \right]$$

Смотря на эту формулу вряд ли можно подумать, что стало легче. Но заметим, что

$$q(x_{t-1} | x_t, x_0) = \frac{q(x_{t-1}, x_t, x_0)}{q(x_t, x_0)} = \frac{q(x_0) q(x_{t-1} | x_0) q(x_t | x_{t-1})}{q(x_0) q(x_t | x_0)} = \frac{q(x_{t-1} | x_0) q(x_t | x_{t-1})}{q(x_t | x_0)}$$

Все распределения в оставшейся формуле являются нормальными из пространства одной размерности. Исходя из вида плотности многомерного нормально распределения можно понять, что $q(x_{t-1} | x_t, x_0)$ тоже является нормальным распределением.

В новой формуле также появилось интересное распределение $q(x_T | x_1)$, которое отвечает за семплирование на целых T шагов вперед, хотя до этого мы семплировали от предыдущего шага к следующему.

Из того, как был введен марковский процесс, можно понять, что семплирование на несколько шагов вперед будет суммой случайных нормальных векторов:

$$q(x_t | x_1) = \mathcal{N} \left(x_t \mid \prod_{i=1}^t \sqrt{1 - \beta_i} x_0, \left(1 - \prod_{i=1}^t \beta_i \right) I \right)$$

Поэтому $q(x_T | x_1)$ тоже является нормальным распределением.

Возвращаясь к вопросу о простоте новой формулы лосса, все распределения в KL-дивергенциях оказываются нормальными, а для их вычисления известна аналитическая формула, поэтому проблем при подсчете не возникнет.

Зато можно показать, что засчет того, что в новой оценке часть матожидания уже вычислена, у нее будет меньше дисперсия. Более того, теперь нет необходимости семплировать всю траекторию для обучения.

Действительно, засчет возможности семплирования на несколько шагов вперед, можно вместо всей траектории брать случайные моменты. Новый алгоритм обучения описывается так:

- Семплируем момент времени $t \sim U[1, \dots, T]$ и $x_t \sim q(x_t | x_0)$
- Считаем градиент и обновляем параметры θ :

$$\nabla_{\theta} \left(\text{KL} \left(q(x_{t-1} | x_t, x_0) \parallel p_{\theta}(x_{t-1} | x_t) \right) \right)$$

Таким образом, получили сильно более эффективное обучение. Однако при этом генерация все равно будет работать в T шагов, что ощутимо медленнее, чем в тех же GANax.

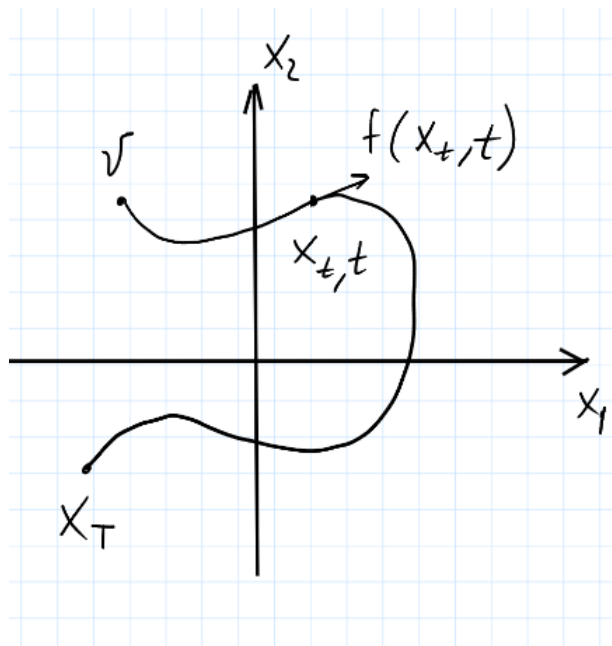
Для того, чтобы ускорить именно инференс, нужно перейти в непрерывное время.

1.2. Обыкновенные дифференциальные уравнения

Введем $x_t \in \mathbb{R}^n$, время $t \in [0, T]$ и направление dx_t , которое в каждый момент времени выражается через векторное поле, а также начальное условие. Более формально:

$$\begin{cases} dx_t = f(x_t, t) dt \\ x_0 = v \in \mathbb{R}^n \end{cases}$$

Такое уравнение задает некоторую траекторию x_t до момента времени T :



Используя схему интегрирования Эйлера, можно численно посчитать для любого начального условия v всю траекторию x_t до момента времени T :

$$x_{t+h} = x_t + hf(x_t, t) + o(h) \approx x_t + hf(x_t, t)$$

Хочется описывать диффузию с непрерывным временем как непрерывную траекторию, которая из картинки переходит в шум. К сожалению, одних обыкновенных дифференциальных уравнений для этого не хватает, так как в диффузии траектории на самом деле случайные.

1.3. Стохастические дифференциальные уравнения

Для этого будем работать со стохастическими дифференциальными уравнениями, которые преобразовывают во времени не сам x_t , а его распределение (распределение траекторий).

Формальная постановка очень похожа на предыдущую:

$$\begin{cases} dx_t = f(x_t, t) dt + g(t) dW_t \\ x_0 \sim p_0 - \text{начальное распределение} \end{cases}$$

То есть в отличие от прошлой постановки, в каждый момент времени x_t дополнительно движется в случайном направлении dW_t , где W_t – стандартный винеровский процесс.

С помощью той же схемы интегрирования Эйлера, получаем

$$x_{t+h} \approx x_t + hf(x_t, t) + \sqrt{h} g(t) \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, I)$$

Решением будет являться распределение на траектории $x_t \sim p_t$.

1.4. Обращение времени

Процесс зашумления картинки на самом деле можно представить в виде стохастического уравнения выше, но нас интересует именно обратный процесс.

Другими словами, мы хотим получить уравнение, в котором время движется в обратную сторону, но для каждого момента времени t распределение на траектории совпадает с распределением в исходном уравнении.

Теорема об обращении времени в таких уравнениях говорит, что обратный процесс таков:

$$\begin{cases} dx_t = \left[f(x_t, t) - g^2(t) \nabla_x \log p_t(x_t) \right] dt + g(t) dW_t \\ x_T \sim p_T \end{cases}, \quad t \in [T, 0]$$

$\nabla_x \log p_t(x_t)$ называется **скор-функцией**.

Если нам уже известно решение исходного уравнения (распределение траектории), то можно посчитать обратное уравнение:

$$x_{t-h} = x_t - hf(x_t, t) + hg^2(t) \nabla_x \log p_t(x_t) + \sqrt{h} g(t) \varepsilon_t$$

Таким образом, зная скор-функцию, можно развернуть любое стохастическое уравнение.

1.5. Обобщение DDPM на непрерывное время

Воспользуемся стохастическими дифференциальными уравнениями, чтобы записать нашу модель в непрерывном времени.

Прямой процесс имеет следующий вид:

$$\begin{cases} dx_t = -\frac{1}{2}\beta(t)x_t dt + \sqrt{\beta(t)} dW_t \\ x_0 \sim p_{data} \end{cases}, \quad t \in [0, T]$$

Разворачивая этот процесс, получаем обратный:

$$\begin{cases} dx_t = \left[-\frac{1}{2}\beta(t)x_t - \beta(t)\nabla_x \log p_t(x_t) \right] dt + \sqrt{\beta(t)} dW_t \\ x_T \sim \mathcal{N}(0, I) \end{cases}, \quad t \in [T, 0]$$

Мы научились считать обратный процесс, но только при известной скор-функции, которой на самом деле пока что нет. Оказывается, ее можно учить, и этого хватает.

Более того, $\mu_\theta(x_t, t)$ из DDPM – это и есть скор-функция с точностью до константного множителя. То есть оптимизируя DDPM, оптимизируется и скор-функция.

Вспомним, что мы все это заварили с целью ускорить процесс генерации. Заметим, что теперь при инференсе можно использовать адаптивный шаг h из схемы Эйлера, а не идти единичными дискретными шагами как это было исходно.

Например, в зависимости от имеющегося времени и ресурсов, из момента времени T в момент времени 0 при генерации можно сделать пару больших или много маленьких шагов. Оба способа будут работать и вернут картинку, но появляется трейдофф между временем генерации и качеством получаемых семплов.

1.6. Denoising score matching

При оптимизации скор-функции, нам нужно знать распределение траектории. Для этого, как упоминалось выше, будем приближать скор-функцию нейросетью:

$$s_\theta(x_t, t) \approx \nabla_x \log p_t(x)$$

В качестве лосса логично оптимизировать разницу

$$\int_{[0, T]} \int_{\mathbb{R}^n} p_t(x_t) \|s_\theta(x_t, t) - \nabla_x \log p_t(x_t)\|_2^2 dx dt \rightarrow \min_\theta$$

Но мы не умеем считать $\nabla_x \log p_t(x_t)$, а как раз хотим его аппроксимировать.

На помощь приходит невероятный факт, формулирующий в точности эквивалентную задачу оптимизации, которую мы уже можем решить:

$$\int_{[0, T]} \int_{\mathbb{R}^n} p_t(x_t) \|s_\theta(x_t, t) - \nabla_x \log p_t(x_t)\|_2^2 dx dt \rightarrow \min_\theta$$

$$\Longleftrightarrow$$

$$\mathbb{E} \left[\|s_\theta(x, t) - \nabla_x \log p_t(x_t | \mathbf{x}_0)\|_2^2 \right] \rightarrow \min_\theta$$

А $\nabla_x \log p_t(x_t | \mathbf{x}_0)$ уже зависит не от p_{data} , которое мы не знаем, а от одного семпла из p_{data} , которые у нас есть (обучающая выборка). Наконец, обучение выглядит следующим образом:

- Семплируем момент времени $t \sim U[0, T]$
- Берем обучающий объект x_0 из датасета и семплируем $x_t \sim p(x_t | x_0)$
- Считаем $\nabla_x \log p_t(x_t | x_0)$ и делаем градиентный шаг s_θ

Выводы

Диффузия является областью, где очень много математики, которая действительно работает. В отличие от GANов, где есть теоретическое обоснование, но на практике приходится отходить от него и придумывать тысячи ухищрений, в диффузионных моделях математический прогресс напрямую влияет на качество получаемых моделей.